LOOPGEN: TRAINING-FREE LOOPABLE MUSIC GENERATION

Davide Marincione* **Giorgio Strano*** **Donato Crisostomi**
Roberto Ribuoli **Emanuele Rodolà**
Sapienza University of Rome
{marincione, strano}@di.uniroma1.it

ABSTRACT

Loops–short audio segments designed for seamless repetition–are central to many music genres, particularly those rooted in dance and electronic styles. However, current generative music models struggle to produce truly *loopable* audio, as generating a short waveform alone does not guarantee a smooth transition from its endpoint back to its start, often resulting in audible discontinuities. We address this gap by modifying a non-autoregressive model (`MAGNeT`) to generate tokens in a circular pattern, letting the model attend to the beginning of the audio when creating its ending. This inference-only approach results in generations that are aware of future context and loop naturally, without the need for any additional training or data. We evaluate the consistency of loop transitions by computing token perplexity around the seam of the loop, observing a 55% improvement. Blind listening tests further confirm significant perceptual gains over baseline methods, improving mean ratings by 70%. Taken together, these results highlight the effectiveness of inference-only approaches in improving generative models and underscore the advantages of non-autoregressive methods for context-aware music generation.

github.com/gladia-research-group/loopgen
gladia-research-group.github.io/loopgen-demo

1. INTRODUCTION

Loops play a critical role in music production across a broad range of genres, from hip-hop to electronic dance music. By definition, a loop is a segment of audio that can be repeated indefinitely without noticeably jarring transitions between consecutive repetitions. These short segments function as building blocks in many compositions, providing rhythmic and harmonic foundations that can be layered, remixed, and manipulated. Indeed, entire online platforms (e.g., Splice [1]) revolve around sharing and curating loops, underscoring their commercial and creative significance in contemporary music-making.

However, despite their ubiquity in practice, loops remain an underexplored challenge for generative music models. The primary issue lies in the disconnect between *generating a short audio sample* and *ensuring that it loops correctly*. Many existing generative approaches focus on producing samples that sound coherent when played from start to finish [1, 2, 3, 4, 5, 6], but they do not explicitly consider the transition point from the end of the sample back to its beginning. As a result, naive repetition of these segments often yields abrupt discontinuities, limiting their practical utility for musicians and producers who rely on seamless repetition.

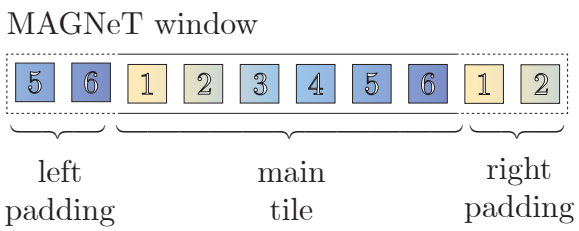


Figure 1. Our proposed circular padding framework for loopable sample generation.

In this paper, we introduce a loop-aware generation framework that modifies the *iterative inference* of a non-autoregressive (NAR) model to produce seamless loops. Concretely, we adopt a *circular padding* strategy, replicating partial portions of the loop at both ends of the generation window, so that the model attends to the loop's beginning while generating its ending (Figure 1). This ensures a smooth endpoint-to-onset transition, effectively creating "bridging tokens" that align the tail of the sample with its onset. Our method can be used in two ways: (1) to generate entire loopable segments from scratch, or (2) to refine the end of an existing audio sample so that it loops seamlessly. Additionally, we implement a *beat-aware* technique that constrains the total length of the loop to align with musical bars, further promoting coherent repetition.

To evaluate this approach, we propose a new perplexity-based metric that quantifies the harshness of the cut at the seam of the loop. Intuitively, if the loop boundary is truly coherent, then it should not be perceived as irregular or dissonant, neither for a human listener, nor for an audio model, as a well-trained network should roughly match human perception.

Our contributions are:

- **Loop-Aware Generation via Audio Tiling**: We propose a new inference procedure that can be applied to a NAR music transformer, such as `MAGNeT`,

* denotes equal contribution.
[1] https://splice.com/

to create seamlessly loopable audio samples. We call this method, and the resulting model, `LoopGen`.

- **Perplexity-Based Seamlessness Metric**: We introduce a metric to quantify the quality of loop boundaries, retrieving the entropy in the "seam" region of a track.
- **Empirical Validation and Code Release**: We show that our system yields superior results according to both quantitative metrics and human listening tests, and we release our code to foster future research on the generation of musical loops.

2. RELATED WORK

Recent advances in **music generation** leverage large-scale transformer-based architectures, which have displaced traditional recurrent neural networks for long-range sequence modeling. Pioneering systems like *MuseNet* [7] and *Music Transformer* [8] showed that attention-based models [9] could capture rich compositional structure in symbolic formats. More recently, state-of-the-art audio transformer models such as *MusicGen* [1] and [3, 10, 11, 12], have demonstrated high-quality generation of waveforms, capable of handling minutes-long clips conditioned on text or user-provided melodies.

Another wave of expressive and accurate models has come with the advent of diffusion models [13] such as *AudioLDM* [4] and [5, 6, 14, 15, 16]. Their application has also reached audio and music and, in this, they are giving high quality results on-par with the transformer models.

Parallel decoding has emerged as a promising alternative to speed up generation. *MAGNeT* [2] employs a single non-autoregressive transformer, such as those used in NLP tasks [17, 18], to predict masked audio tokens iteratively, showing that a well-designed masking and rescoring strategy can close the quality gap with autoregressive baselines at a fraction of the inference cost. *VampNet* [19], another non-autoregressive approach, introduces inpainting capabilities and partial rewriting to refine music segments, including short repeated "vamps," demonstrating promise for loop-centric workflows. Likewise, *SoundStorm* [20] applies a bidirectional transformer on semantic tokens for efficient speech and music synthesis, further illustrating the viability of non-autoregressive methods for audio.

Loopable music remains comparatively underexplored. *LoopNet* [21] specifically targets the generation of seamless music loops, but it is tied to a limited dataset of loops which falls short of the general-purpose of free-form approaches. Other work has focused on *symbolic loops* in MIDI [22, 23], proposing architectures that ensures segments are musically consistent when repeated. However, these methods are intrinsically different from raw audio tokens; MIDI loops require explicit pitch and instrument representations, which do not transfer to audio-generation tasks. Recently, *DITTO* [24] introduced an inference-time optimization that allows fine-grained control, including looping, over text-to-music diffusion models. While *DITTO* is notable for its high output quality, it requires memory comparable to a full fine-tuning, and it slows down inference by a factor of $\sim 100\times$.

Finally, loopable *media* generation is being tackled in computer vision with tiling techniques. Models like *TileGAN* [25] and [26, 27] synthesize textures or images that repeat edge-to-edge without seams. While these visual approaches share the overarching idea of boundary alignment, they do not directly address audio continuity or musical structure.

In this paper, we build on *MAGNeT*'s non-autoregressive design to propose an inference-time approach for loopable music generation, avoiding additional training or data requirements. By treating time in a "circular" manner, our method enforces continuity at the loop boundary, substantially improving perceptual seamlessness in raw audio.

3. BACKGROUND

3.1 MAGNeT's inference

Unlike typical NAR models, `MAGNeT`'s inference does not emit all output tokens in a single inference pass. Instead, it develops the audio clip iteratively. In particular, at each iteration, `MAGNeT`:

1. Generates logits for each empty token in the sequence.
2. Samples a value for each token.
3. Selects the highest scoring tokens, and marks them as fixed.
4. Re-empties the remaining non-fixed tokens and, if no empty tokens are left, terminates; otherwise, it starts the next iteration.

Following [2], we use `MAGNeT`'s own logits to select the tokens for the first iteration. `MAGNeT`'s inference can be viewed as a generalization of autoregressive inference: rather than receiving a continuous sequence of tokens and outputting the next token, `MAGNeT` operates on a set of empty and non-empty tokens, filling multiple empty positions in each iteration. Thanks to its non-causal self-attention, `MAGNeT` can condition its outputs on both past *and* future tokens, ensuring coherent generation across boundaries. This property makes `MAGNeT` (and similar non-autoregressive models) well-suited for loop creation, because the model can naturally attend to the loop's start while generating its end, thereby facilitating a smoother, more seamless transition.

3.2 Rescoring

`MAGNeT` [2] also proposes a variant that linearly interpolates the probabilities given by its own logits, with those of another audio model, such as `MusicGen` [1], to calculate the scores for the selection procedure. This results in a trade-off between higher quality and increased computational cost, as calculating another model's probabilities requires running it alongside `MAGNeT`.

3.3 Hybrid MAGNeT

As noted in [2], when presented with short audio tracks, MAGNeT produces continuations which, on average, sound better than samples created from scratch. Knowing this, we test both generating samples from scratch, and continuations of clips produced with MusicGen.

4. METHOD

While our framework is, in principle, applicable to *any* NAR model that generates audio iteratively, we choose MAGNeT [2] as our base system because it is currently the state-of-the-art in NAR music generation.

We adapt MAGNeT's iterative inference to create a "circular" context around the central segment of tokens that will form our final loop. By replicating partial portions of this loop segment at the beginning and end of the generation window, MAGNeT can attend to the loop's start when predicting its end, and vice versa. We refer to the central segment as the *main loop tile*.

4.1 Iterative overview

MAGNeT generates audio tokens in several iterations. Each iteration partially fills an overall generation window of length L. We isolate a specific subrange of length c near the center of this window to become our main loop tile. The remaining space on the left and right is filled with copies of the tile's end or beginning, respectively, thus forming a circular context.

4.2 Inference algorithm

At **initialization**, we start with an empty (or partially filled) window of length L. In the middle of this window, we mark out c consecutive positions as the main loop tile.

1. Filling the Context. Before calling MAGNeT, and before each inference step, we copy:

- The *ending* of the main tile into the *left* side of the window, so that the first tokens of the tile can "see" what happens at the end of it.
- The *beginning* of the main tile into the *right* side of the window, so that the last tokens of the tile can "see" its start.

This ensures a fully circular arrangement: the model effectively observes how the loop's end meets its beginning.

2. MAGNeT Inference. We run MAGNeT on the entire window of length L. Because MAGNeT uses non-causal (bidirectional) attention, tokens in the main tile can be conditioned on both the left-side copy (its own end) and the right-side copy (its own start).

3. Token Selection. At the end of each iteration, only tokens *within the main tile* are considered for finalizing. We keep those that MAGNeT assigns the highest probability (e.g., top-k or threshold-based), marking them as fixed (i.e., no longer empty in subsequent iterations). The rest are reset to empty.

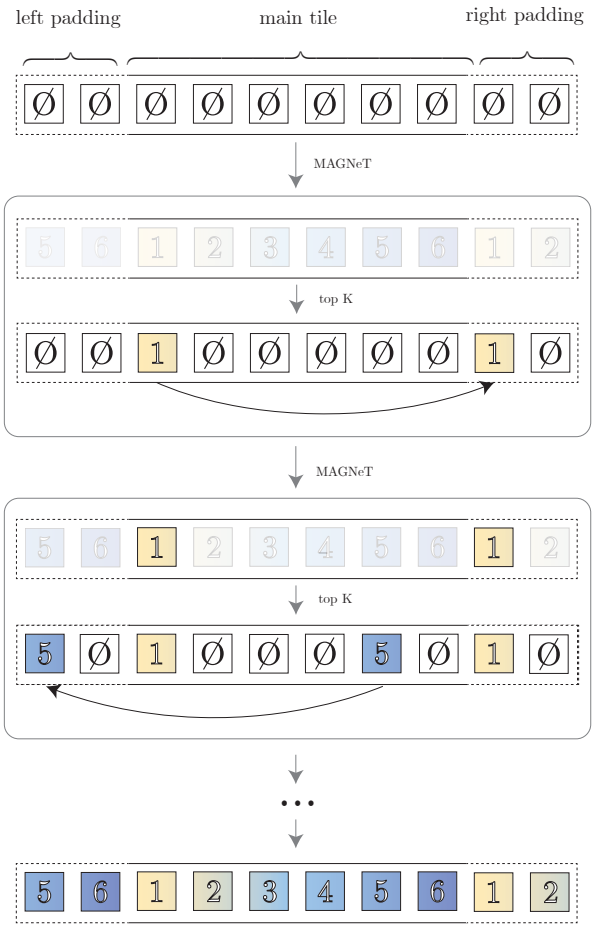


Figure 2. Diagram of our approach. The central *main tile* represents the final audio segment to be looped. At each inference step, only the top-k samples are maintained and reflected in the tiles. This circular padding lets MAGNeT attend to both the start and end of the tile simultaneously, ensuring a smooth transition at the loop boundary.

4. Repeat until completion. We move on to the next inference iteration, going back to step **2**, until the entirety of the *main tile* is filled.

5. Extract the final loop. Once the iteration limit is reached or all main-tile tokens are fixed, the algorithm stops. The central c tokens (our main loop tile) are extracted as the final result. Repeating this tile end-to-start yields a seamless loop.

4.3 Hybrid variant

MAGNeT often produces higher-quality audio when continuing from a given prompt rather than generating entirely from scratch [2]. To take advantage of this, we first generate an audio segment C with MusicGen, empirically set to half the desired final clip length. For instance, if the final clip is intended to last 10s, we let MusicGen produce the first 5s and then provide these tokens as a partially filled main tile. This approach forces the model to generate a coherent continuation of the high-quality prompt, ensuring

the ending transitions seamlessly to the beginning. Empirically, we observe that this hybrid version surpasses samples generated without an audio prompt in terms of semantic variety, objective audio quality, and musical coherence.

4.4 Signature-aware length control

A well-formed loop often sounds most musical when it aligns with full bars (e.g., 2 or 4 bars of consistent tempo). Generating loops of arbitrary length may create awkward breaks if, for instance, the tempo does not fit integer bar divisions.

To mitigate this, we use the current state-of-the-art beat-extraction system, `beat_this` [28] on the initial audio prompt C, to identify:

- The average duration between beats, $\delta \approx 60/\text{BPM}$.
- The median number of beats per bar, μ.

We use these to compute the duration of a bar that the prompt C implies. We want the duration of the entire loop l to be *a)* an exact multiple (or submultiple) of the duration of a bar; *b)* constrained in a time interval $[\alpha, \beta]$. To achieve this, we repeatedly double or halve the initial candidate length l until it fits these constraints.

Algorithm 1 Beat Alignment algorithm

Require: Audio clip C, min/max duration α and β, preferred number of bars n

```
B, D ← detected beats/downbeats in beat_this(C)
δ ← median time elapsed between Bs        ▷ akin to 60/BPM
μ ← median #beats between Ds              ▷ bar of the clip
l ← nμδ                                   ▷ duration of n bars
while l < α ∨ l > β do
    if l < α then
        l ← 2l
    else
        l ← l/2
    end if
end while
if l ∈ [nμδ/4, 4nμδ] then                 ▷ Too far from n bars?
    return l                              ▷ Return if sufficiently close
else
    return ∅                              ▷ Otherwise abort (try another C)
end if
```

When used in combination, tiled generation and the beat alignment Algorithm 1 produce loops that not only have smooth seam transitions but also respect musical structure. This results in clips that are more naturally *loopable* for applications like music production, live performance, or any setting in which tightly aligned repeating segments are required.

5. EVALUATION METRICS

When assessing looped music, a clip that sounds fine in a single pass may still have an abrupt transition when it repeats. Standard metrics such as FAD [29], which measure the overall distributional similarity between generated and real audio using neural embeddings, may miss such artifacts. Because FAD operates over full-length clips, it can overlook localized issues such as discontinuities at the loop boundary. To address this, we propose a metric that focuses directly on the transition seam.

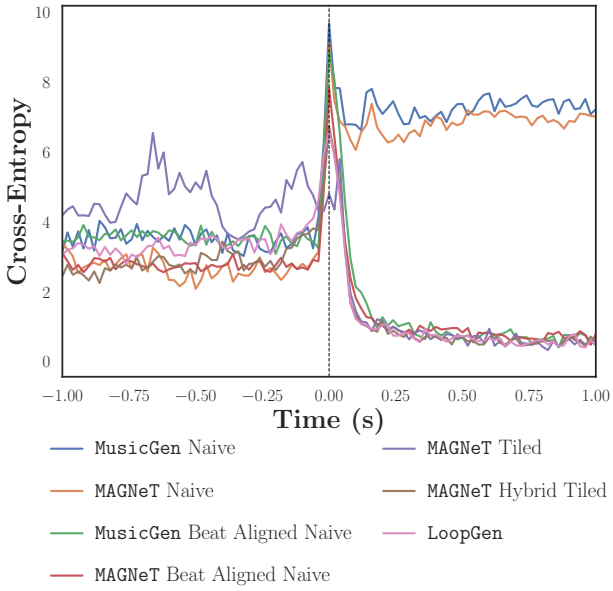


Figure 3. Average cross entropy of `MusicGen` around the seam (highlighted with a dashed line) for different models/variants.

5.1 Seam perplexity

To automatically assess the continuity of the loop around the seam, we adapt the idea of *perplexity* from language modeling. Let us assume that we have a well-trained music generation model (such as `MusicGen`) that can estimate probabilities for each token (or audio frame) in a music clip. While traditional perplexity sums over all tokens in a clip, we focus exclusively on the seam, that is, the transition point where the end meets the beginning, where loop artifacts are most likely to occur.

5.1.1 *Cross-entropy and perplexity*

First recall that, for a sequence $X = (x_1, x_2, \ldots, x_T)$, the average cross-entropy $H(X)$ is:

$$H(X) = -\frac{1}{T}\sum_{i=1}^{T} \ln \mathcal{M}(x_i). \tag{1}$$

Intuitively, if $\mathcal{M}$ assigns higher probability to each token, the cross-entropy will be smaller, indicating better alignment between model and data. From cross-entropy, we derive the perplexity $\mathcal{P}(X)$, a standard measure of how well a model predicts a sequence:

$$\mathcal{P}(X) = \exp\big(H(X)\big) = \exp\Big(-\frac{1}{T}\sum_{i=1}^{T} \ln \mathcal{M}(x_i)\Big). \tag{2}$$

A lower perplexity value indicates that $\mathcal{M}$ finds the sequence more predictable (or more likely).

5.1.2 *Seam perplexity*

While global perplexity focuses on the entire clip, loop artifacts, as in Figure 3, occur specifically at the boundary where the clip wraps around. To isolate how the model perceives that transition, we compute *seam perplexity* on a short window around the boundary.

Let us be given N generated clips $\{X^{(k)}\}$. Each $X^{(k)}$ has length T, and we identify a seam boundary at index $b^{(k)}$. We then define a window of size W immediately following $b^{(k)}$, i.e., the tokens

$$X^{(k)}_{\text{seam}} = \left(x^{(k)}_i : i \in [b^{(k)},\, b^{(k)} + W - 1]\right). \quad (3)$$

The average cross-entropy of the seam tokens in $X^{(k)}$ is:

$$H_{\text{seam}}(X^{(k)}) = -\frac{1}{W} \sum_{i=b^{(k)}}^{b^{(k)}+W-1} \ln \mathcal{M}\left(x^{(k)}_i\right). \quad (4)$$

Finally, the seam perplexity is the exponential of the mean seam cross-entropy across all N clips:

$$\text{Seam Perplexity} = \exp\left(\frac{1}{N} \sum_{k=1}^{N} H_{\text{seam}}\left(X^{(k)}\right)\right). \quad (5)$$

A low seam perplexity indicates that the seam is "easy" for a strong reference model to predict, suggesting a smooth transition. Conversely, a high value suggests abrupt discontinuities or other artifacts at the loop boundary.

6. EXPERIMENTS

In the following, all generated tracks are conditioned with the same set of 100 textual prompts, and `MAGNeT`'s iterations are set to $\langle 100, 50, 10, 10\rangle$ for each of the 4 codebooks from `EnCodec` [30] respectively. Textual prompts are generated automatically via a LLM, some of them are (e.g.):

(1) "*A high-energy EDM track with a powerful drop and sidechain compression*"

(2) "*An Irish folk dance tune with energetic fiddle and bodhrán drum*"

6.1 Evaluating models

6.1.1 *Baselines*

For both `MAGNeT` and `MusicGen`, two baseline solutions are formulated: (i) **Naive**, a sample is generated and repeated, without further processing, and (ii) **Beat-Aligned (BA) Naive**, a sample is generated, ran through Algorithm 1 to cut them at a musically-valid length, and repeated.

6.1.2 *Our techniques*

From the contributions of this paper, three models are evaluated: (i) **Tiled**, samples are generated via the tiled generation technique described in Section 4.2, (ii) **Hybrid Tiled**, samples are generated with the same tiled technique, but starting from an audio prompt generated by `MusicGen` (ref. Section 4.3), and (iii) **Beat Aligned Tiled**, which uses the same technique as **Hybrid Tiled**, but with the additional application of the beat-alignment algorithm described in Section 4.4. This latter variant is our best performing model, and what, going forward, we call `LoopGen`. For each variant, both Seam Perplexity and `fadtk`'s [31] FAD (with embeddings from both VGGish[32] and CLAP [33] over the FMA-Pop [34] dataset) are computed.

6.2 Hyperparameter search

The most important hyperparameters for the samples' quality we identify are classifier-free guidance (λ) and the rescoring coefficient (ω). The former controls how much a model should adhere to the conditioning information given (in our case, the textual prompt), instead of following the emerging sample. In `MAGNeT`'s original paper, the authors find that the best FAD is reached with $\lambda = 10.0$ (linearly decreasing to $\lambda = 1.0$ as the iterations pass) but, as the tiling constraint might increase the contextual information that the model can gain from the input, we verified that a lower coefficient translates into more organic generations.

The rescoring coefficient ω, instead, controls the interpolation coefficient introduced in Section 3.2. When $\omega = 0$, rescoring is not applied, when 1, only `MusicGen`'s probabilities are used. We test therefore our algorithm with multiple coefficients ranging from 0 to 1. A reasonable value for the cfg was chosen to be $\lambda = 5.0$; on the other hand, the rescoring was chosen through a thorough search conducted on both `MAGNeT` **Tiled** and `MAGNeT` **Hybrid Tiled**, generating 100 10-seconds samples for each model. Our results, presented in Table 1, empirically show the best rescoring to be $\omega = 0.5$.

It is worth noting that the **Hybrid** version of the model consistently achieves better FAD scores, but worse perplexity. The better FAD score can be clearly attributed to the initial prompt generated by `MusicGen`, which consistently surpasses `MAGNeT`'s audio quality. This hybrid combination of different models is also the reason for the increase in perplexity, since the final generation consists of a concatenation of tokens sampled from different distributions.

Model	Variant	ω	$\text{FAD}_{\text{vggish}}$ (↓)	Seam Perplexity (↓)
`MAGNeT`	Tiled	0.0	3.05	23.88 ± 5.40
`MAGNeT`	Tiled	0.25	3.22	25.17 ± 5.35
`MAGNeT`	Tiled	0.50	3.51	$\mathbf{18.15 \pm 3.53}$
`MAGNeT`	Tiled	0.75	3.97	24.55 ± 5.43
`MAGNeT`	Tiled	1.0	4.35	25.42 ± 4.86
`MAGNeT`	Hybrid Tiled	0.0	2.97	39.30 ± 7.21
`MAGNeT`	Hybrid Tiled	0.25	2.99	47.72 ± 10.11
`MAGNeT`	Hybrid Tiled	0.50	2.98	44.42 ± 9.33
`MAGNeT`	Hybrid Tiled	0.75	3.00	43.93 ± 8.39
`MAGNeT`	Hybrid Tiled	1.0	**2.93**	41.74 ± 9.05

Table 1. Rescoring experiments ($\lambda = 5.0$)

6.3 Final results

Below, we present our final results across six baselines and our three novel models, generated with the same previous 100 textual prompts, but 30 seconds long. **Tiling** models, using the technique described in Section 4.2, exhibit significantly lower Seam Perplexity compared to their non-tiled counterparts, though at the cost of a weaker FAD score. However, `LoopGen`, leveraging both the **Hybrid** approach (Section 4.3) and Algorithm 1, achieves the best FAD score among all models. This improvement comes with a slight increase in Seam Perplexity, as previously discussed.

Despite this minor trade-off in perplexity, `LoopGen` substantially outperforms baseline solutions, offering a more musically pleasing output due to its alignment with rhythmically meaningful cut points (Algorithm 1). This results in tracks that maintain better musical coherence compared to the standard **Tiled** model.

Table 2 presents the evaluation metrics, and the distribution of Seam Perplexity values is visualized in Figure 4.

Model	Variant	$\text{FAD}_{\text{vggish}}$ (↓)	FAD_{CLAP} (↓)	Seam Perplexity (↓)
`MAGNeT`	Vanilla	3.36	0.33	—
`MAGNeT`	Naive	3.36	0.35	1549.06 ± 556.03
`MAGNeT`	Beat Aligned Naive	3.34	0.34	153.22 ± 47.69
`MusicGen`	Vanilla	2.81	0.32	—
`MusicGen`	Naive	2.81	0.33	2512.39 ± 903.16
`MusicGen`	Beat Aligned Naive	2.86	0.33	507.07 ± 163.67
`MAGNeT`	Tiled	4.30	0.51	$\mathbf{56.17 \pm 11.78}$
`MAGNeT`	Hybrid Tiled	2.98	0.33	94.41 ± 25.77
`MAGNeT`	`LoopGen`	**2.80**	**0.31**	84.29 ± 22.66

Table 2. Main experiments' evaluation metrics. For each model, we compute the FAD with VGG-ish and CLAP embeddings using FMA-pop as a reference dataset. For reference, we also compute FAD scores for both `MAGNeT` and `MusicGen`'s standard, non-looping, generations).

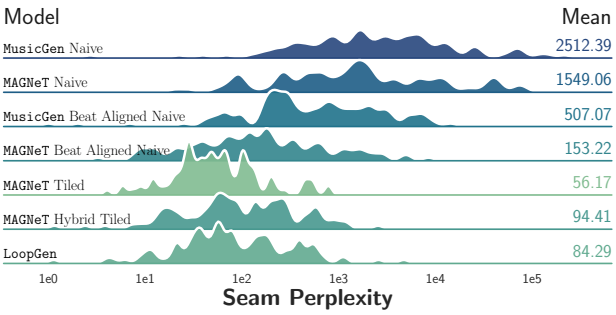


Figure 4. **Seam Perplexity** distribution of the considered models (lower is better).

6.4 Human evaluation

Using the previous setups, we prepare a set of: 100 10-seconds clips from `LoopGen` (ours), and 100 10-seconds clips from `MAGNeT` Hybrid Naive (without Tiling-generation, baseline). We select the latter model because it is the most similar to ours, without any of the modifications introduced in this paper. This ensures a fair comparison, with the primary expected difference being seamlessness. The clips are chosen to be 10 seconds long for ease of listening.

With this set of samples, we conduct a blind listening experiment with a group of users. Each volunteer listens to up to 30 randomly selected clips (15 from our model, 15 from the baseline) and rates the perceptibility of the seam on a Likert scale (1 = Evident cut, 5 = Imperceptible cut). In total, we collect 506 data points s from 18

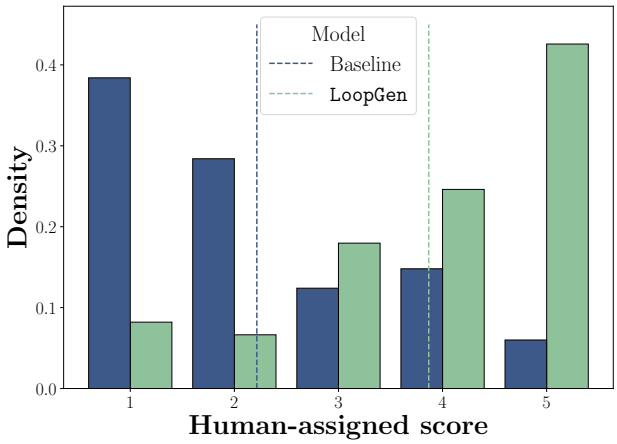


Figure 5. Distribution of perceptibility ratings, comparing `LoopGen` with the baseline. Lines are model's mean.

listening sessions. Computing each user's average rating of each model; we run a paired t-test such that

$$H_0 \equiv \mu_{\texttt{LoopGen}} = \mu_{\text{baseline}} \tag{6}$$

This yields $t(17) = 12.21$, $p < 10^{-9}$, providing overwhelming evidence against H_0. Furthermore, the effect size is large ($d = 2.88$), confirming a very strong evidence that our technique substantially reduces the perceptibility of the seam, as can also be seen in Figure 5.

7. CONCLUSIONS

With this research, we have introduced a novel inference-only approach for generating loopable music, leveraging a simple "circular" padding scheme within MAGNeT's non-autoregressive framework to ensure seamless boundaries. Our experiments demonstrated clear gains in loop continuity, validated both by a new perplexity-based seam metric and by human listening tests. The whole procedure does not require additional training or specialized loop datasets. By aligning loop length to musical beats, the generated audio segments more naturally fit common compositional structures, further improving their usability in practice. Overall, this work underscores the potential of lightweight, inference-time solutions for enhancing generative music models.

8. ACKNOWLEDGMENTS

This work is supported by Sapienza University of Rome via the Seed of ERC grant "MINT.AI", cup B83C25001040001. Furthermore, we thank all of the participants in the human evaluation test.

9. REFERENCES

[1] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and controllable music generation," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[2] A. Ziv, I. Gat, G. Le Lan, T. Remez, F. Kreuk, J. Copet, A. Défossez, G. Synnaeve, and Y. Adi, "Masked audio generation using a single non-autoregressive transformer," in *The Twelfth International Conference on Learning Representations*, 2024.

[3] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," *arXiv preprint arXiv:2005.00341*, 2020.

[4] H. Liu, Z. Chen, Y. Yuan, X. Mei, X. Liu, D. Mandic, W. Wang, and M. D. Plumbley, "AudioLDM: Text-to-audio generation with latent diffusion models," *Proceedings of the International Conference on Machine Learning*, pp. 21 450–21 474, 2023.

[5] F. Schneider, O. Kamal, Z. Jin, and B. Schölkopf, "Moûsai: Text-to-music generation with long-context latent diffusion," 2023. [Online]. Available: https://arxiv.org/abs/2301.11757

[6] S. Forsgren and H. Martiros, "Riffusion - stable diffusion for real-time music generation," 2022. [Online]. Available: https://riffusion.com/about

[7] O. Goren, E. Nachmani, and L. Wolf, "A-muze-net: Music generation by composing the harmony based on the generated melody," 2021. [Online]. Available: https://arxiv.org/abs/2111.12986

[8] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. Shazeer, I. Simon, C. Hawthorne, A. Dai, M. Hoffman, M. Dinculescu, and D. Eck, "Music transformer: Generating music with long-term structure," 2019. [Online]. Available: https://arxiv.org/abs/1809.04281

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[10] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi, M. Sharifi, N. Zeghidour, and C. Frank, "Musiclm: Generating music from text," 2023. [Online]. Available: https://arxiv.org/abs/2301.11325

[11] S. Vasquez, S. Vasquez, M. Lewis, M. Lewis, M. Lewis, M. Lewis, M. Lewis, and M. Lewis, "Melnet: A generative model for audio in the frequency domain," *arXiv: Audio and Speech Processing*, 2019.

[12] J. Gardner, S. Durand, D. Stoller, and R. Bittner, "Llark: A multimodal instruction-following language model for music," *Proc. of the International Conference on Machine Learning (ICML)*, 2024.

[13] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 6840–6851.

[14] J. Nistal, M. Pasini, C. Aouameur, M. Grachten, and S. Lattner, "Diff-a-riff: Musical accompaniment co-creation via latent diffusion models," 2024. [Online]. Available: https://arxiv.org/abs/2406.08384

[15] Z. Evans, C. Carr, J. Taylor, S. H. Hawley, and J. Pons, "Fast timing-conditioned latent audio diffusion," 2024. [Online]. Available: https://arxiv.org/abs/2402.04825

[16] G. Mariani, I. Tallini, E. Postolache, M. Mancusi, L. Cosmo, and E. Rodolà, "Multi-source diffusion models for simultaneous music generation and separation," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=h922Qhkmx1

[17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4171–4186.

[18] J. Li, T. Tang, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "Elmer: A non-autoregressive pre-trained language model for efficient and effective text generation," 2022.

[19] H. F. F. Garcia, P. Seetharaman, R. Kumar, and B. Pardo, "Vampnet: Music generation via masked acoustic token modeling," in *Ismir 2023 Hybrid Conference*, 2023.

[20] Z. Borsos, M. Sharifi, D. Vincent, E. Kharitonov, N. Zeghidour, and M. Tagliasacchi, "Soundstorm: Efficient parallel audio generation," 2023.

[21] P. Chandna, A. Ramires, X. Serra, and E. Gómez, "Loopnet: Musical loop synthesis conditioned on intuitive musical parameters," in *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 3395–3399.

[22] G.-Y. Chen and V.-W. Soo, "Controllable music loops generation with midi and text via multi-stage cross attention and instrument-aware reinforcement learning," in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024, p. 6851–6859.

[23] S. Han, H. R. Ihm, M. Lee, and W. Lim, "Symbolic music loop generation with neural discrete representations," in *International Society for Music Information Retrieval Conference*, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:251493133

[24] Z. Novack, J. McAuley, T. Berg-Kirkpatrick, and N. J. Bryan, "Ditto: Diffusion inference-time t-optimization for music generation," 2024.

[25] A. Frühstück, I. Alhashim, and P. Wonka, "Tilegan: synthesis of large-scale non-homogeneous textures," *ACM Trans. Graph.*, vol. 38, pp. 58:1–58:11, 2019.

[26] C. Rodríguez-Pardo and E. Garces, "Seamlessgan: Self-supervised synthesis of tileable texture maps," *IEEE Trans. Vis. Comput. Graph.*, vol. 29, pp. 2914–2925, 2023.

[27] O. Madar and O. Fried, "Tiled diffusion," 2024. [Online]. Available: https://arxiv.org/abs/2412.15185

[28] F. Foscarin, J. Schlüter, and G. Widmer, "Beat this! accurate beat tracking without dbn postprocessing," in *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, San Francisco, CA, United States, Nov. 2024.

[29] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, "Fréchet audio distance: A metric for evaluating music enhancement algorithms," 2019. [Online]. Available: https://arxiv.org/abs/1812.08466

[30] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.

[31] A. Gui, H. Gamper, S. Braun, and D. Emmanouilidou, "Adapting frechet audio distance for generative music evaluation," in *Proc. IEEE ICASSP 2024*, 2024. [Online]. Available: https://arxiv.org/abs/2311.01616

[32] S. Hershey, S. Chaudhuri, D. P. W. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold, M. Slaney, R. J. Weiss, and K. Wilson, "Cnn architectures for large-scale audio classification," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 131–135.

[33] Y. Wu*, K. Chen*, T. Zhang*, Y. Hui*, T. Berg-Kirkpatrick, and S. Dubnov, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," in *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP*, 2023.

[34] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "FMA: A dataset for music analysis," in *18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017. [Online]. Available: https://arxiv.org/abs/1612.01840

A. SEAM PERPLEXITY'S ERROR MARGINS

In various tables, we present the values of our Seam Perplexity as center $\pm$ standard error. Because perplexity is the exponentiation of average cross-entropy, it is impossible to actually compute error margins directly. To obtain these values, we start from Equation (4) and compute for each dataset of samples $\mathbf{X} = \{X^1, \ldots, X^N\}$ the mean cross-entropy:

$$\mu_{\mathbf{X}} = \tfrac{1}{N} \sum_{k=1}^{N} H_{\text{seam}}\big(X^{(k)}\big) \tag{7}$$

and standard deviation

$$\sigma_{\mathbf{X}} = \sqrt{\frac{1}{N-1} \sum_{k=1}^{N} \big(\mu_{\mathbf{X}} - H_{\text{seam}}\big(X^{(k)}\big)\big)^2}. \tag{8}$$

We then compute the 95% confidence intervals for the cross-entropy

$$\left[\mu_{\mathbf{X}} - 1.96 \tfrac{\sigma_{\mathbf{X}}}{\sqrt{N}}, \mu_{\mathbf{X}} + 1.96 \tfrac{\sigma_{\mathbf{X}}}{\sqrt{N}}\right], \tag{9}$$

and transform them into exponential space

$$\left[l = \exp\left(\mu_{\mathbf{X}} - 1.96 \tfrac{\sigma_{\mathbf{X}}}{\sqrt{N}}\right), r = \exp\left(\mu_{\mathbf{X}} + 1.96 \tfrac{\sigma_{\mathbf{X}}}{\sqrt{N}}\right)\right]. \tag{10}$$

Finally, we calculate the provided values as

$$\text{center} = \tfrac{1}{2}(l + r), \qquad \text{standard error} = \tfrac{1}{2}(r - l). \tag{11}$$

This approach differs from the common method of showing a value with error margins, where the error is modeled as Gaussian, and the center value is assumed to be the empirical mean of the measured quantity. In this case, however, since the perplexity operation itself is computed as the exponentiation of its mean, it would be impossible to calculate a symmetric Gaussian error margin directly (not without running calculations on multiple folds of the data).

B. 10 SECONDS EXPERIMENTS

During development, we also explored the same final experiments seen in the main article (Table 2) with the 10 seconds variant of `MAGNeT`. The results of these experiments are detailed in Table 3 and visualized in Figure 6. Notably, the **Seam Perplexity** exhibits a significant change with this modification. While it is unclear whether this change is solely attributable to the different models, the shorter track length, or a combination thereof, we empirically observed no discernible perceptual difference in the seamlessness of the 10-second and 30-second samples.

Model	Variant	$\text{FAD}_{\text{vggish}}(\downarrow)$	$\text{FAD}_{\text{CLAP}}(\downarrow)$	Seam Perplexity ($\downarrow$)
`MAGNeT`	Vanilla	3.05	0.39	—
`MAGNeT`	Naive	3.02	0.31	310.21 ± 98.33
`MAGNeT`	Beat Aligned Naive	3.03	0.35	202.43 ± 67.23
`MusicGen`	Vanilla	3.28	0.41	—
`MusicGen`	Naive	3.21	0.34	529.79 ± 167.87
`MusicGen`	Beat Aligned Naive	3.24	0.31	302.88 ± 79.91
`MAGNeT`	Tiled	3.51	0.40	18.15 ± 3.53
`MAGNeT`	Hybrid Tiled	2.98	0.33	44.42 ± 9.33
`MAGNeT`	`LoopGen`	2.95	0.33	60.85 ± 15.24

Table 3. 10 seconds versions of main experiments' evaluation

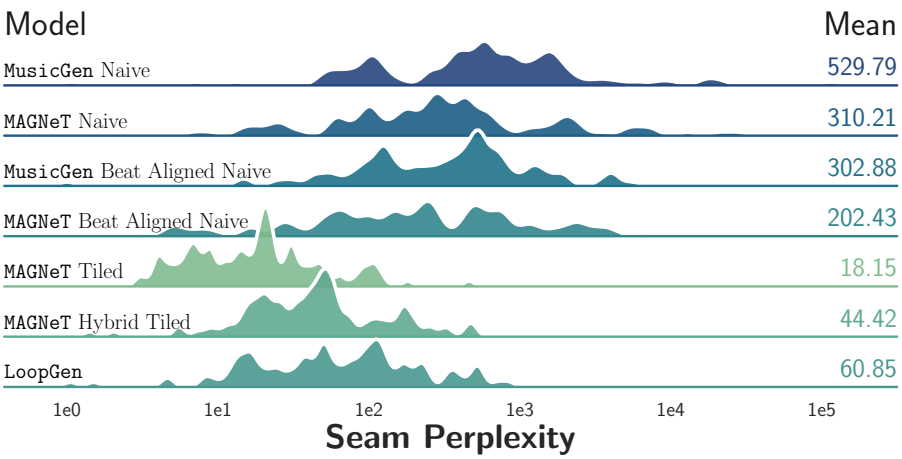


Figure 6. **Seam Perplexity**'s distributions for 10 seconds samples (lower is better).

C. BEAT ALIGNMENT ALGORITHM VISUALIZATION

To further illustrate the inner workings of our beat alignment algorithm, we provide a visual representation in Figure 7. This figure highlights the key aspects of the algorithm as applied to a typical waveform. Specifically, it showcases how the algorithm ensures that audio segments are cut at musically meaningful locations, preventing awkward segment lengths that could disrupt the perception of rhythmic continuity.

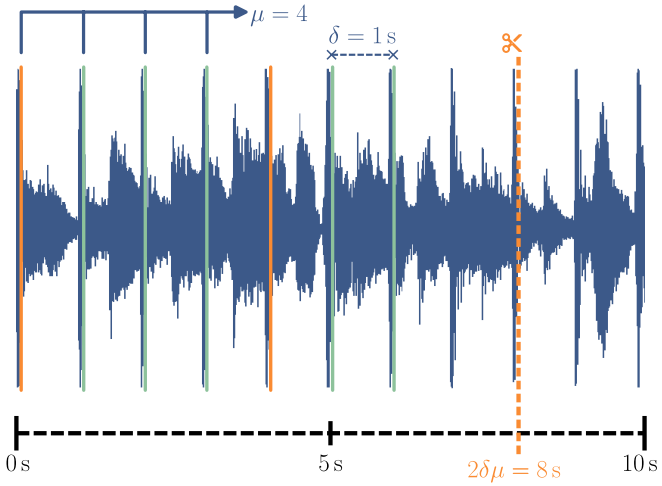


Figure 7. Visualization of our beat alignment algorithm (Algorithm 1). Downbeats are shown in orange, and beats in light green. If the waveform were cut at the 10-second mark, the resulting unit would be 2.5 bars long—musically unpleasing when repeated. Instead, our algorithm ensures cuts occur at a whole-bar length or, if not possible, at a multiple.

D. MAGNET'S INFERENCE

In the original paper of `MAGNeT` [2], the authors provide a detailed pseudocode of the inference procedure, we show the same in Figure 8, with our added lines highlighted. It can be seen how our modification is actually pretty agnostic to the model behaviour and, it can be argued, that it may indeed be applied to other models which act in a similar manner.

```python
def loopgen_MAGNeT_generate(B: int, T: int, c: int, text: List, s: int, model: nn.Module,
                    rescorer: nn.Module, mask_id: int, tempr: float, w: float):

    # Start from a fully masked sequence
    gen_seq = torch.full((B, T), mask_id, dtype=torch.long)

    n_spans = T #// span_len    To fine-control which samples are picked
    spans_shape = (B, n_spans)
    span_scores = torch.zeros(spans_shape, dtype=torch.float32)

    # Addition 1.  Calculate limits of main tile
    l = min((T - c) // 2, c)
    r = T - c - l

    # Run MAGNeT iterative decoding for 's' iterations
    for i in range(s):
        mask_p = torch.cos((math.pi * i) / (2 * s))
        n_masked_spans = max(int(mask_p * n_spans), 1)

        # Masking
        masked_spans = span_scores.topk(n_masked_spans, dim=-1).indices
        mask = get_mask(spans_shape, masked_spans)
        gen_seq[mask] = mask_id

        # Addition 2.  Copy main tile to padding areas
        if r > 0:
            gen_seq[..., :l] = gen_seq[..., -l-r:-r]
            gen_seq[..., -r:] = torch.cat([gen_seq[..., l:-r]] * (r // c + 1), -1)[..., :r]

        # Forward pass
        logits, probs = model.compute_predictions(gen_sequence, text, cfg=True, temperature=tempr)

        # Classifier free guidance with annealing
        cfg_logits = cfg(mask_p, logits, annealing=True)

        # Sampling
        sampled_tokens = sample_top_p(probs, p=top_p)

        # Place the sampled tokens in the masked positions
        mask = gen_seq == mask_id
        gen_seq = place_sampled_tokens(mask, sampled_tokens[..., 0], gen_seq)

        # Probs of sampled tokens
        sampled_probs = get_sampled_probs(probs, sampled_tokens)
        if rescorer:
            # Rescoring
            rescorer_logits, rescorer_probs = rescorer.compute_predictions(gen_seq, text)
            rescorer_sampled_probs = get_sampled_probs(rescorer_probs, sampled_tokens)

            # Final probs are the convex combination of probs and rescorer_probs
            sampled_probs = w * rescorer_sampled_probs + (1 - w) * sampled_probs

        # Addition 3.  Give no probability to padding areas
        sampled_probs[..., :l] = 0
        sampled_probs[..., -r:] = 0

        # Span scoring - max
        span_scores = get_spans_scores(sampled_probs)

        # Prevent remasking by placing -inf scores for unmasked
        span_scores = span_scores.masked_fill(~spans_mask, -1e5)

    return gen_seq
```

Figure 8. Modified `MAGNeT` inference procedure, stylized for ease of reading (wouldn't actually run correctly in its current form), our added lines are highlighted in light-brown.